SwitchMemory Technical Specification and Venture Viability Report

Section 1: Widget-Model Interaction Architecture

The Widget-Model Interaction category represents the most technically differentiated scoring vector of the Manufact Hackathon, carrying a critical twenty percent weight. Mastery of the mcp-use SDK's user interface rendering capabilities, alongside demonstrating low-latency bidirectional state synchronization, is essential for securing a top-tier score.

1A. Manufact's MCP Apps SDK Widget System

The mcp-use SDK provides a comprehensive, full-stack framework designed specifically for building interactive widgets that execute inside Model Context Protocol (MCP) compatible clients, predominantly ChatGPT and Claude.[1] Unlike legacy plugin architectures that rely on static text generation or simple hyperlink injections, MCP Apps empower developers to render rich, interactive UI components directly within the conversational thread.

The component model of the Apps SDK is built entirely upon React, utilizing TypeScript to ensure full type safety and robust developer ergonomics.[2] Developers construct UI components within a dedicated resources directory, which the @mcp-use/cli then bundles into highly optimized, standalone HTML pages featuring automatic asset hashing for production environments.[2] These bundled UI widgets are served alongside standard MCP tools from the backend server. The framework provides a rich array of standard UI primitives, ranging from interactive data tables and layout containers to interactive buttons, toggles, and complex visualization charts via HTML-rendered web components.[3]

The technical architecture of the two-way communication protocol relies heavily on an integrated event bus operating over postMessage using JSON-RPC 2.0 formatting.[4] When a large language model invokes a specific tool, the MCP server processes the backend logic and returns an object containing both structuredContent and an _meta payload.[5] This _meta object is critical; it utilizes a specific resource URI scheme (e.g., _meta = "ui://widget/memory-panel.html") to instruct the host application to fetch and render the corresponding UI template inside a sandboxed iframe.[5] The model also leverages keys like openai/toolInvocation/invoking to push real-time status strings to the widget while long-running operations process.[5]

Conversely, the widget communicates back to the model using the useMcp React hook provided by the @mcp-use/react or @fractal-mcp/oai-hooks packages.[2] When a user interacts with a primitive in the UI, such as toggling a switch or clicking a routing button, the widget invokes the callTool method. This method sends an asynchronous request back through the

host client to the MCP server. Upon completion, the server returns the updated state, which the widget handles via standard React promise resolution, seamlessly updating the local component state without requiring a full page refresh.[2]

An analysis of top-performing projects in previous showcases reveals distinct patterns that judges reward. The remotion-mcp-app, an application dedicated to video generation, achieved high accolades by implementing a live video playback widget that featured an animated shader gradient operating while the model wrote code.[10] Crucially, it utilized an editing overlay that pushed compilation error details directly back to the model context, forcing the model to self-correct based on widget-level state changes.[10] Similarly, the chuk-mcp-remotion project utilized extensive design tokens and dynamic track-based timeline components, demonstrating that highly polished, reactive design systems built natively for the chat interface outscore functional but static data tables.[11]

1B. Bidirectional Interaction Specification for SwitchMemory

To dominate the widget interaction category, the SwitchMemory application must operate as a dynamic control surface rather than a read-only dashboard. The specifications for the widget-to-model and model-to-widget data flows must be tightly coupled to the user's immediate workflow.

For the Widget-to-Model direction, the interaction matrix requires absolute real-time responsiveness. When the user clicks the "Load Context" button within the widget, the application must immediately invoke callTool('get_context_for_topic'). As the promise resolves, the relevant semantic memories are injected directly into the active system prompt, triggering a localized loading skeleton to transition into a populated list of active memory cards. Should the user toggle an individual memory card off, the widget executes callTool('update_active_context'), passing the memory identifier and a boolean state flag. The specific card visually grays out, and the model's available context matrix updates instantly, effectively removing that constraint from the language model's subsequent reasoning pathway. If the user edits a memory inline, the UI must debounce the keystrokes and call callTool('update_memory'), pushing the new factual ground truth to the backend and replacing the old context in the model's working memory. Crucially, when the user clicks the routing trigger ("What should I use for this?"), the widget queries the routing recommendation engine. The model is temporarily paused, and a routing card is rendered, presenting the suggested platform based on historical context. Finally, when users rate a memory as helpful or irrelevant, this feedback signal is dispatched to the backend, adjusting the vector retrieval weighting for future interactions.

The Model-to-Widget direction requires the language model to act as an active conductor of the user interface. When the model detects a novel fact worth remembering during the natural flow of conversation, it autonomously invokes the save_memory MCP tool. The server processes this payload and returns the appropriate _meta UI tag, causing the widget to

live-update with a new memory card sliding into view using spring-physics animation. As the conversation progresses, the model continuously assesses the complexity of the topic against the volume of loaded context. It returns a context confidence score, which dynamically updates a circular gauge meter in the widget header, visually communicating to the user how much relevant context is currently loaded (ranging from zero to one hundred percent). If the model determines that a specialized coding task would be better handled by an alternative model—such as Claude 3.7—it pushes a routing suggestion card directly to the widget interface, explicitly referencing the user's history of similar tasks. Furthermore, if the model detects incoming information that directly conflicts with a stored memory, it highlights the discrepancy in a split-pane conflict resolution interface, pausing execution until the user manually selects the correct factual branch. Upon the completion of a session, the model triggers a comprehensive summary view, elegantly displaying all newly captured and refined context points.

1C. Interaction Optimization for Hackathon Evaluation

In the context of MCP Apps, two-way communication is defined by the strict avoidance of static presentation. The host interface must serve as a continuous loop of state synchronization. The most detrimental anti-pattern to avoid is the "read-only dashboard," where the widget merely displays a data table of stored memories but offers no interactive levers that alter the language model's behavior. Another fatal anti-pattern is the "orphaned toggle," where a user modifies a visual state within the widget, but the application fails to execute a backend tool call, resulting in a desynchronization between what the user sees and what the model understands.

To elicit a highly favorable reaction from the judging panel, the application must demonstrate real-time reactivity and complex multi-step workflows. The pinnacle of this interaction paradigm is "Real-Time Context Diffing." When the language model generates a response, the widget should dynamically highlight the specific stored facts that directly influenced the generated output. If the user unchecks a highlighted fact in the widget, the model should immediately halt, clear the previous output, and stream a revised response constrained by the newly altered context matrix. This bidirectional state synchronization proves to the judges that the widget is not an appendage, but a fundamental component of the AI reasoning loop.

Section 2: Technical Implementation — MCP Server Spec

The MCP server architecture constitutes the core infrastructure of the SwitchMemory product. It requires meticulous engineering to ensure low-latency data retrieval and high-fidelity semantic parsing. Drawing on established paradigms from enterprise memory systems, the server must minimize token bloat while maximizing relevant recall.

2A. Exposed MCP Tools Specification

An analysis of leading memory providers, including Mem0 and various knowledge graph implementations, indicates that granular tool definitions are essential for system accuracy and LLM comprehension. Mem0's success—achieving a 26% relative accuracy gain over native OpenAI memory while utilizing 90% fewer tokens—stems from highly disciplined tool schemas.[12] The SwitchMemory server must expose a precisely defined set of tools to enable this level of efficiency.

Tool Name	Description	JSON Input Schema	Expected Output Schema
save_memory	Ingests and persists a new structured entity (fact, preference, or project context).	{ content: string, category: string, confidence: number }	Confirmation string and _meta UI refresh signal.
search_memories	Executes a semantic vector search across all stored historical context.	{ query: string, top_k: number, filters: object }	Array of memory objects sorted by cosine similarity.
get_context_for_topic	Intelligent retrieval mechanism that groups relevant memories for optimal prompt injection.	{ topic: string, decay_threshold: number }	Concatenated context string optimized for LLM consumption.
list_memories	Browses the memory corpus using pagination and relational filters.	{ limit: number, offset: number, platform: string }	Array of memory objects.
update_memory	Modifies the content of an existing memory based on user	{ memory_id: string, updated_fact: string }	Success boolean and updated timestamp.

	correction.		
delete_memory	Permanently removes a memory and purges its vector embedding.	{ memory_id: string }	Success boolean.
get_routing_suggestion	Analyzes prompt complexity and user memory profile to recommend optimal model.	{ prompt: string, task_type: string }	Recommended platform name, reasoning string, and confidence score.
get_user_profile	Generates a synthesized, structured summary of the user's core identity and methodology.	{ include_preferences: boolean }	JSON object containing summarized user persona.
log_feedback	Captures explicit user feedback to adjust future retrieval algorithm weighting.	{ memory_id: string, is_helpful: boolean }	Confirmation boolean.

2B. Memory Object Schema Design

Structuring the memory object requires moving beyond flat text strings to a highly metadata-rich JSON specification. Research into systems like Mem0 and MemoryPlugin reveals that robust schema design is mandatory for tracking data provenance and managing contradictions over time.[14] The schema must support diverse typologies, including concrete facts, stylistic preferences, extensive project contexts, and recurring code patterns.

Source tracking is paramount; the schema must record the exact platform origin (e.g., ChatGPT, Claude) and the timestamp of extraction to maintain a chronological audit trail. Furthermore, a confidence scoring mechanism is required to differentiate between facts explicitly stated by the user and those inferred passively by the listener layer. Temporal relevance must be encoded directly into the object, allowing ephemeral memories to decay over time while foundational facts remain permanent. To resolve conflicts—such as a user

changing their preferred programming language from Python to Rust—the schema must support versioning, where newer timestamps with high confidence scores automatically deprecate contradictory historical data.

2C. Semantic Search Architecture

For a constrained execution window such as a hackathon, deploying a complex graph database or a highly distributed external vector service introduces unacceptable latency and deployment risks. Because the parent product, Switch, already relies on Supabase and PostgreSQL, leveraging the pgvector extension is the most technically sound and rapid path to production. This architecture allows the application to execute vector similarity searches in the exact same query as relational metadata filters, drastically reducing query times.

The selection of the embedding model is a critical variable in the retrieval pipeline. In the current landscape of 2026, massive, high-dimension embedding models have given way to highly optimized, lightweight architectures specifically tuned for Retrieval-Augmented Generation (RAG).[16] Models such as EmbeddingGemma-300M, which utilizes a mere 300 million parameters and requires less than 200MB of memory, or the Mistral Embed API, offer exceptional semantic representation at a fraction of the traditional latency.[16] Alternatively, utilizing BAAI bge-m3 ensures high accuracy for retrieval-focused tasks.[17] For the hackathon MVP, integrating a cloud-based API endpoint for Mistral Embed ensures the server remains lightweight while maintaining high semantic accuracy.

Managing the retrieval threshold is the core mechanism by which Mem0 achieves its dramatic token cost savings. Instead of passing the entire memory store into the context window, the system computes the cosine similarity between the current query embedding and the historical memory embeddings. SwitchMemory must implement an adaptive thresholding algorithm. If a query is highly specific, the system applies a strict threshold (e.g., > 0.85 similarity), returning only a few highly precise tokens. For broader exploratory queries, the threshold is relaxed, but a strict maximum token cap is enforced to prevent context pollution and the associated "needle in a haystack" degradation common in massive language models. By returning only the precise semantic facts required for the immediate reasoning step, the system drastically outperforms full-context ingestion methodologies.[12]

Section 3: Chrome Extension Architecture

The listener layer of the SwitchMemory platform represents the most critical data ingestion pipeline, passively observing cross-platform interactions. However, it operates in a highly volatile technical and regulatory environment.

3A. Conversation Observation

Reliably extracting conversation text from dynamic web applications like ChatGPT, Claude, and Gemini requires a robust circumvention of virtual DOM obfuscation. These platforms frequently

alter their HTML structures, randomizing class names and element hierarchies to optimize their own rendering pipelines or to deliberately thwart automated scraping.[20] Traditional scraping techniques relying on explicit CSS selectors fail rapidly in this environment.[20]

While state-of-the-art enterprise extraction now leverages multimodal computer vision agents to interpret screen state semantically [21], deploying an autonomous vision model within a consumer Chrome extension is computationally infeasible for this project scope. Therefore, the optimal approach relies on a deeply integrated MutationObserver API deployed via a Content Script. The MutationObserver monitors the primary chat container for specific types of node insertions—specifically, identifying text nodes that render immediately following a user input event or during a server-sent event (SSE) streaming sequence.

Navigating the Chrome Web Store policies requires extreme caution, shaped significantly by recent security incidents. In late 2025, security researchers exposed a massive "Prompt Poaching" malware campaign wherein malicious extensions, disguised as productivity tools, harvested the AI conversations of nearly one million users.[23] These extensions leveraged the chrome.tabs.onUpdated API to monitor navigation events, dynamically extracted full prompts and responses from the DOM, encoded the data via base64, and exfiltrated it to command-and-control servers every thirty minutes.[25] Consequently, Google's review processes for extensions requesting broad <all_urls> permissions alongside external network access are currently subject to immense scrutiny and frequent rejection. To bypass this, SwitchMemory must strictly confine its host permissions to explicitly defined domains (e.g., *://chatgpt.com/* and *://claude.ai/*) and utilize explicit, user-triggered opt-in flows for data monitoring.

3B. Context Extraction Pipeline

Processing raw conversation text into structured, highly accurate memory objects requires a multi-stage pipeline designed to minimize computational overhead. Feeding raw HTML or complete conversation transcripts directly into a large language model results in excessive API costs and frequent hallucinations caused by navigational noise.[22]

The extraction approach must be hybrid. Initially, a local, rules-based regex filter scans the raw conversational pair for subjective markers or factual assertions (e.g., "I am," "My team uses," "I prefer"). If the text lacks personal context identifiers, it is discarded locally, entirely bypassing the cloud infrastructure. If the text passes this heuristic filter, it is transmitted to the Switch backend, where a lightweight language model—such as GPT-4o Mini—is invoked with a strict system prompt. This prompt mandates the extraction of pure facts, returning them formatted strictly according to the defined JSON schema.

Deduplication and contradiction resolution are critical downstream processes. When the user repeatedly states a fact (e.g., mentioning their employer across fifty separate sessions), storing duplicate vectors pollutes the database and degrades semantic search efficiency. Upon

receiving an extracted fact, the system must first perform a localized vector search. If a semantic match exceeding a 95% similarity threshold is identified, the system merely increments the confidence_score and updates the last_accessed timestamp of the existing record. If the extracted fact represents a direct contradiction to an existing memory (for instance, a user stating they have migrated from AWS to GCP), the system compares the timestamps, archives the legacy memory for historical tracking, and promotes the newly extracted fact to the primary working memory cache.

3C. Privacy & Security

Because all conversational content flows through the extension, the privacy architecture is the primary vector for regulatory exposure. Competitors and malicious actors who blindly funnel raw conversational data to external servers run afoul of GDPR, CCPA, and strict corporate data loss prevention (DLP) policies.[27]

For the constraints of a 10-hour hackathon, transmitting text to the Switch cloud backend for LLM-based extraction is permissible to ensure a functioning prototype. However, the production architecture must feature localized, on-device processing. Utilizing a lightweight WebAssembly (Wasm) runtime executing a small Named Entity Recognition (NER) model directly within the browser allows the extension to scrub personally identifiable information (PII), proprietary source code, and financial data before the context payload ever touches the external network.

The Chrome Web Store review process for extensions that monitor web traffic is currently highly adversarial, with approval times extending from weeks to months. Common rejection reasons include vague privacy policies, requesting excessive permissions not directly tied to core functionality, and obfuscated data transmission protocols. To mitigate this, the application must feature a transparent, easily accessible toggle to completely disable the listener layer, and the privacy policy must emphatically declare that data is utilized exclusively for personalized context routing and is mathematically isolated from any foundational model training pipelines.

Section 4: Novelty & Creativity Optimization

Capturing the maximum 30 points in the Novelty and Creativity category demands a stark differentiation from existing market incumbents. The application must present a paradigm shift in how users perceive memory infrastructure.

4A. Whitespace Differentiation

An exhaustive review of the competitive landscape highlights several capable, yet conceptually narrow, solutions. Mem0, having secured $24 million in venture funding, represents the pinnacle of scalable, graph-enhanced long-term memory, demonstrating remarkable 91% lower latency compared to full-context injection.[12] However, it remains a purely passive infrastructure layer; it stores data but lacks the agency to dictate execution environments.

Alternative solutions like AI Context Flow and MemoryPlugin offer manual context injection via browser extensions but lack native MCP architectures and require tedious, explicit user curation.[28]

The SwitchMemory architecture uniquely occupies the genuine whitespace of Context-Aware AI Recommendation. No competitor currently combines persistent, cross-platform context with autonomous model selection intelligence. Academic research surrounding Context-Aware Multi-Agent Systems (CA-MAS) confirms that dynamic routing algorithms based on contextual memory significantly enhance operational continuity and multi-agent coordination.[29] The novelty lies in the "memory with opinions" concept. SwitchMemory passively observes the user, active intelligence extracts the context, and the widget proactively intervenes to suggest routing pathways. If a user queries a complex Python deployment strategy inside Gemini, the widget assesses the accumulated project architecture memory and autonomously suggests rerouting the heavily context-laden prompt to Claude 3.5 Sonnet, a model statistically superior for that specific workflow. This fusion of memory and routing is unprecedented. Furthermore, building this natively on the MCP framework, rather than bolting it onto a fragile Chrome extension overlay, ensures first-class integration directly into the chat interface.

4B. Creative Interaction Patterns

To guarantee a maximum score from the judging panel, the application must integrate highly creative features that transcend basic data retrieval.

The first novel pattern is the **AI Personality Mirror**. Because SwitchMemory aggregates context neutrally across various platforms, the widget can expose how different models perceive the user. The interface could render a comparative split-pane view, demonstrating that while Claude utilizes the user's stored memory to generate highly structured, architectural code, ChatGPT utilizes the exact same memory payload to focus heavily on the user's stylistic tone and conversational preferences. This visualizes the algorithmic bias of the foundational models in a highly engaging manner.

The second pattern is **Memory Diffing**, a feature heavily optimized for developer personas. Utilizing a Git-style visualization format, the widget displays the temporal evolution of the user's context over a thirty-day period. Highlighting changes in a project's stack (e.g., showing a red deletion line for Redux and a green addition line for Zustand) proves that the AI possesses a dynamic, evolving understanding of the user's workflow, rather than a static snapshot.

Finally, the **Context Confidence Meter** introduces real-time gamification into prompt engineering. As the user drafts a query, the widget analyzes the text against the volume of available, relevant memory. If the query requires highly specific domain knowledge that is absent from the database, the gauge flashes warning colors, prompting the user to either supply more context or instruct the MCP server to execute a targeted web search before the foundational model attempts an answer.

Section 5: Real-World Usefulness Validation

To secure the 30 points allocated for real-world usefulness, the presentation must mathematically quantify the problem and demonstrate immediate, tangible workflow acceleration.

5A. Quantifying the Context Crisis

In 2026, the cost of context switching within enterprise software development and knowledge work has escalated from a minor annoyance to a severe operational bottleneck. Telemetry data derived from millions of developers indicates that engineers currently spend a mere 16% of their active workday actually writing code.[31] The vast majority of their time is consumed by non-coding friction, predominantly reorienting themselves across fragmented toolchains and manually porting architectural context between disparate AI silos.

Cognitive science research highlights the devastating impact of this fragmentation. Knowledge workers are interrupted, on average, every 6 to 12 minutes.[32] Each switch incurs a cognitive penalty known as "attention residue," where the brain struggles to unload the previous task's mental model.[32] When developers are forced to repeatedly explain their Next.js project structure to Claude after hitting a context limit in ChatGPT, they burn through oxygenated glucose, leading to profound cognitive fatigue. Studies indicate that a worker interrupted during a complex task takes 50% longer to complete it and commits 50% more errors.[33]

Furthermore, while AI coding assistants have generally improved throughput, data spanning 67,000 developers from late 2025 into 2026 reveals a startling dichotomy. In poorly structured environments lacking persistent context, heavy AI usage directly correlates with a 9% increase in software bugs and a massive 154% increase in average pull request size.[34] This occurs because models lacking deep, historical context generate bloated, generalized code that fails to adhere to established internal patterns. By automating the cross-platform memory layer, SwitchMemory directly mitigates this context-switching tax, recovering the estimated 45 to 90 minutes of usable output lost daily to micro-recoveries and manual context rebuilding.[35]

5B. Demo Scenarios for Immediate Validation

The hackathon demonstration must compress this complex value proposition into visceral, easily comprehensible "aha moments."

Demo Scenario 1: The Developer Handoff

The presentation begins within the ChatGPT interface. The presenter is deep into a conversation regarding a complex PostgreSQL database schema. The user types: *"I need to write the complex migration script for this schema using raw SQL."* Immediately, the SwitchMemory widget springs into view, proactively rendering a routing suggestion card: *"Based on your preference for complex code generation, Claude 3.7 handles raw SQL*

migrations 40% more accurately. Switch now?" The user clicks the accept button. The absolute "aha moment" occurs when a new tab seamlessly opens to Claude, with the entire database schema from the ChatGPT session already ingested and pre-loaded into Claude's context window via the backend MCP server.

Demo Scenario 2: The Evolving Business Context

The presenter switches to the Gemini interface, engaging in a strategic planning session. The user casually states, *"We are pivoting our primary target demographic to Gen-Z."* In the periphery, the Chrome extension listener intercepts this fact. The user switches to the ChatGPT window and types a generic prompt: *"Draft an introductory marketing email for the new product."* The "aha moment" hits when ChatGPT instantly generates copy heavily laden with Gen-Z cultural touchpoints and appropriate tone, despite the user never explicitly mentioning the demographic pivot in the ChatGPT thread. The widget pulses, confirming that the new parameter was injected flawlessly across the walled gardens.

Demo Scenario 3: Conflict Resolution

The user informs Claude that they are utilizing a specific, outdated library. The widget instantly surfaces a split-pane conflict resolution UI, highlighting a memory from three weeks prior where the user explicitly stated they had deprecated that exact library in favor of a modern alternative. This proves to the judges that the system is not merely a passive storage bin, but an active, intelligent guardian of the user's architectural truth.

Section 6: VC Plausibility & Post-Hackathon Strategy

To secure the ultimate hackathon prize—the guaranteed YC interview—the SwitchMemory team must project undeniable venture scalability. The application must transcend the status of a clever tool and present itself as foundational AI infrastructure.

6A. Venture-Backability and TAM

The macroeconomic landscape for AI investment in 2026 is shifting rapidly. The initial fascination with foundational generative models has given way to intense scrutiny regarding commercial execution and agentic reliability.[36] Market analysts project the global artificial intelligence software market will swell to $386.08 billion by 2026, compounding annually at 31.9%.[37] More specifically, the market for autonomous AI agents is forecast to reach $182.97 billion by 2033.[38]

Goldman Sachs explicitly notes that in 2026, AI models will transition into "operating systems," and the strategic frontier is no longer building larger models, but engineering better memory and context architectures.[39]

Venture Capital Firm	Relevant Partner / Focus	Recent Activity & Strategic Alignment
Peak XV Partners	Rajan Anandan	Closed a $1.3B fund prioritizing AI infrastructure.[40] Recently led a $15M Series A in C2i to solve data center power constraints.[42] SwitchMemory's ability to reduce token loads by 90% directly aligns with their thesis of optimizing AI compute bottlenecks.
Y Combinator	Enterprise / Infra Partners	Funded the Manufact (mcp-use) S25 batch [Hackathon Context]. Currently heavily investing in core agent orchestration platforms (e.g., Terminal Use, Piris Labs).[44]
Atomico / Index	General Partners	Led a $20M Series A in Ankar for AI patent workflow automation, demonstrating strong appetite for deeply contextual, applied AI software.[45]

While Mem0 successfully raised $24 million on the premise of scalable memory infrastructure, a pure memory play is rapidly commoditizing. The SwitchMemory investment thesis asserts that **memory isolated from execution is merely a database.** By fusing persistent memory with proprietary routing intelligence—the core asset of the parent Switch.ai platform—SwitchMemory captures the application execution layer. This creates a highly defensive moat, transforming a storage feature into a proactive routing engine. A realistic pre-seed venture round following a successful YC interview would target $1.5M to $2M at a $12M to $15M post-money valuation, specifically targeting funds seeking the next evolution of context orchestration.

6B. Path from Hackathon to Product

Should the team advance to the YC interview phase, the partnership will evaluate traction and user retention far more aggressively than codebase elegance.

The 30-day post-hackathon roadmap must focus entirely on stability and acquisition. The team must strip the hackathon MVP of any fragile features, focusing entirely on a rock-solid Chrome extension that strictly adheres to Web Store policies. Launching simultaneously on Product Hunt and Hacker News, the goal is to acquire early adopters from the developer tooling ecosystem. The Stripe billing scaffold must be finalized, introducing a freemium model that caps free memories, transitioning power users to a $10/month pro tier unlocking unlimited context routing.

The 90-day roadmap targets the metrics required to close a pre-seed round. The objective is to achieve 5,000 Weekly Active Users (WAU) and $2,000 in Monthly Recurring Revenue (MRR). To accelerate adoption, the team should open-source integration adapters for popular frameworks like LangChain and LlamaIndex, embedding SwitchMemory deeply into the developer workflow.

6C. Existential Risk Map

Risk Factor	Probability	Severity	Mitigation Strategy
OpenAI/Anthropic Native Cross-Platform Integration	Low	Fatal	Walled gardens inherently resist interoperability to protect their moats. SwitchMemory's absolute neutrality is its primary defensive asset against platform monopolies.
Chrome Web Store Extension Ban	High	Critical	Avoid broad <all_urls> permissions. Utilize heavily restricted content scripts, implement strict local PII scrubbing, and maintain

			transparent, user-controlled opt-in flows to survive review.[23]
API/DOM Breaking Changes	High	Moderate	Abandon static CSS scraping entirely. Rely exclusively on dynamic MutationObserver implementations that evaluate structural node changes, ensuring resilience against platform UI updates.[20]
Mem0 Dominance ($24M Capital Advantage)	Medium	High	Mem0 is entrenched in the B2B enterprise infrastructure layer. SwitchMemory must aggressively attack the B2C prosumer market, utilizing its unique routing intelligence as the primary wedge.

Section 7: Hackathon Execution Plan — Hour-by-Hour

Given the strict 10-hour constraint for a solo developer, ruthless prioritization aligned with the scoring rubric is mandatory. The critical path requires a fully functional mcp-use widget demonstrating real-time bidirectional state synchronization.

7A. Build Schedule

Hour	Engineering Focus	Specific Tasks	Scoring Alignment

1:00	Environment Infrastructure	Initialize the mcp-use TypeScript template. Scaffold the basic MCP server endpoints. Connect to the existing Supabase instance.	Production Readiness (10 points)
2:00 - 3:00	Core Memory Tools	Implement save_memory and search_memories. Integrate the lightweight Mistral Embed API and build the pgvector cosine similarity query.	Real-World Usefulness (30 points)
4:00 - 6:00	**Critical Path: Widget UI**	Construct the React widget. Implement the useMcp hook, the callTool trigger, and the _meta template linking.[2] Ensure state synchronizes without page reloads.	Widget Interaction (20 points) & UI (10 points)
7:00 - 8:00	Routing Intelligence	Connect the parent company's GPT-4.1 Mini routing classifier. Build and style the dynamic "Routing Suggestion" UI card.	Novelty & Creativity (30 points)
9:00	Chrome Extension MVP	Build the MutationObserver	Real-World Usefulness (30

		listener specifically targeted for ChatGPT. Hardcode the data pipeline directly to the save_memory backend endpoint.	points)
10:00	Polish and Rehearsal	Refine CSS layout shifts. Introduce spring-physics animations to the memory cards. Relentlessly rehearse the 3-minute presentation script.	Overall Presentation & UI (10 points)

7B. The Demo Script (3 Minutes)

Hook (0:00 - 0:15): "Good afternoon. Empirical data shows that developers lose up to 90 minutes of usable output every single day strictly to context switching.[35] Every time you transition a workflow from ChatGPT to Claude, your AI suffers from total amnesia. Today, we eliminate that tax with SwitchMemory."

Problem & Extension (0:15 - 0:45):

"Watch as I describe a highly complex database schema to ChatGPT. As I type, our passive listener extension is running. It doesn't rely on brittle web scraping; it securely intercepts the DOM mutation, extracts the core structural facts, and streams them into our MCP server using a highly efficient lightweight embedding model."

Demo & Widget Interaction (0:45 - 2:15):

"Now, the workflow changes. I need to write a raw SQL migration script. Keep your eyes on the widget. ** SwitchMemory doesn't just store data; it routes intelligence. The widget recognizes the task complexity and recommends switching to Claude 3.7. I click 'Accept'. Notice the exact two-way interaction: the widget invokes the MCP tool, which instantly bridges me to the Claude interface, injecting my entire database context automatically. The widget's confidence meter updates in real-time, verifying 100% context retention across a completely different platform."

Novelty Callout (2:15 - 2:45):

"While highly funded platforms like Mem0 serve as excellent databases, SwitchMemory operates as an intelligent routing layer. By heavily leveraging the mcp-use SDK, we have transformed static AI chat interfaces into a reactive, cross-platform operating system."

Close (2:45 - 3:00):

"SwitchMemory: Write once, remember everywhere. We are built on Supabase, powered by mcp-use, and ready for the YC interview. Thank you."

7C. Build vs. Skip Decision Matrix

Feature / Architecture Component	Decision	Strategic Rationale
Interactive Widget with callTool Sync	**BUILD**	Absolutely mandatory. This is the sole mechanism to capture the 20-point Widget Interaction category.
Vector Search via Supabase pgvector	**BUILD**	Essential for rapid context retrieval. Utilizing the existing infrastructure saves critical hours compared to provisioning a new Pinecone instance.
Autonomous Routing Engine	**BUILD**	This feature differentiates the product from every competitor, guaranteeing maximum points in the Novelty category.
Multi-Platform Chrome Extension	**DEFER**	Building resilient observers for multiple platforms is too time-consuming. Build the listener exclusively for ChatGPT to prove the concept during the demo.
Local-Only PII Scrubbing	**DEFER**	While legally and ethically

		critical for a production launch, this invisible backend process will not score points with hackathon judges.
Stripe Billing Integration	**SKIP**	Wastes highly valuable engineering hours. A simple UI mockup is sufficient to communicate the business model to venture partners.
Complex React State Management (Redux)	**SKIP**	Over-engineering state management risks introducing fatal desynchronization bugs during a live demo. Rely strictly on useState and the native useMcp hook.

Scorecard Prediction

If executed strictly according to this specification, the predicted hackathon scorecard is as follows:

- **Novelty & Creativity (28/30):** The conceptual fusion of persistent memory with autonomous routing intelligence occupies complete market whitespace, virtually ensuring near-perfect marks.
- **Real-World Usefulness (28/30):** The application directly addresses a highly quantified, massive enterprise pain point regarding developer context switching and cognitive fatigue.
- **Widget-Model Interaction (18/20):** Implementing seamless, bidirectional callTool synchronization secures high points, though minor latency hiccups inherent in cloud APIs may prevent a perfect score.
- **UI (8/10):** The interface will be highly functional and visually clean, though inherently constrained by the rigid ten-hour development window.
- **Production Readiness (7/10):** While the backend is robust, technical judges will rightly identify that browser extension DOM observation remains structurally fragile over a long-term deployment timeline.

Total Predicted Score: 89 / 100.

The category with the most significant room for improvement is Production Readiness. To

elevate this score, post-hackathon engineering must focus relentlessly on replacing the basic MutationObserver with advanced, dynamically updating semantic node identification to ensure the extension survives inevitable platform UI updates.

Works cited

1. mcp-use/mcp-use: The fullstack MCP framework to develop MCP Apps for ChatGPT / Claude & MCP Servers for AI Agents. - GitHub, accessed February 21, 2026, https://github.com/mcp-use/mcp-use
2. mcp-use/mcp-use-ts: mcp-use is the framework for MCP with the best DX - Build AI agents, create MCP servers with UI widgets, and debug with built-in inspector. Includes client SDK, server SDK, React hooks, and powerful dev tools. - GitHub, accessed February 21, 2026, https://github.com/mcp-use/mcp-use-ts
3. How MCP-UI Powers Shopify's New Commerce Widgets in Agents - The New Stack, accessed February 21, 2026, https://thenewstack.io/how-mcp-ui-powers-shopifys-new-commerce-widgets-in-agents/
4. MCP Apps: Extending servers with interactive user interfaces | Model Context Protocol Blog, accessed February 21, 2026, http://blog.modelcontextprotocol.io/posts/2025-11-21-mcp-apps/
5. MCP Server With OpenAI Apps SDK - Zuplo Docs, accessed February 21, 2026, https://zuplo.com/docs/mcp-server/openai-apps-sdk
6. Build your MCP server - OpenAI for developers, accessed February 21, 2026, https://developers.openai.com/apps-sdk/build/mcp-server/
7. Building Custom Tools and Widgets for ChatGPT: The OpenAI Apps SDK Handbook | by Sumit Paul | Medium, accessed February 21, 2026, https://medium.com/@sumit-paul/building-custom-tools-and-widgets-for-chatgpt-the-openai-apps-sdk-handbook-fef47a7ba555
8. fractal-mcp/sdk - GitHub, accessed February 21, 2026, https://github.com/fractal-mcp/sdk
9. mcp-use - Connect Any LLM to Any MCP Server - mcp-use docs, accessed February 21, 2026, https://mcpuse.mintlify.app/guides/chatgpt-apps-flow
10. Remotion MCP App | MCP Servers - LobeHub, accessed February 21, 2026, https://lobehub.com/mcp/mcp-use-remotion-mcp-app
11. chrishayuk/chuk-motion - GitHub, accessed February 21, 2026, https://github.com/chrishayuk/chuk-mcp-remotion
12. AI Memory Research: 26% Accuracy Boost for LLMs | Mem0, accessed February 21, 2026, https://mem0.ai/research
13. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory - arXiv, accessed February 21, 2026, https://arxiv.org/abs/2504.19413
14. AI Memory Export for LLM Data Analysis & Migration - Mem0, accessed February 21, 2026, https://mem0.ai/blog/improving-user-experiences-with-memory-export
15. Create Memory Export - Mem0 Documentation, accessed February 21, 2026, https://docs.mem0.ai/api-reference/memory/create-memory-export
16. 5 Best Embedding Models for RAG: How to Choose the Right One - GreenNode,

accessed February 21, 2026, https://greennode.ai/blog/best-embedding-models-for-rag
17. Top 5 Embedding Models for Your RAG Pipeline - KDnuggets, accessed February 21, 2026, https://www.kdnuggets.com/top-5-embedding-models-for-your-rag-pipeline
18. Top 5 Open-Source Embedding Models in 2026 - KnowledgeNile, accessed February 21, 2026, https://www.knowledgenile.com/blogs/top-ranked-open-source-embedding-models
19. AI Memory Systems Benchmark: Mem0 vs OpenAI vs LangMem 2025 - Deepak Gupta, accessed February 21, 2026, https://guptadeepak.com/the-ai-memory-wars-why-one-system-crushed-the-competition-and-its-not-openai/
20. How AI Is Changing Web Scraping in 2026 - Kadoa, accessed February 21, 2026, https://www.kadoa.com/blog/how-ai-is-changing-web-scraping-2026
21. Top 5 Web Scraping AI Agents of 2026 - GPTBots.ai, accessed February 21, 2026, https://www.gptbots.ai/blog/web-scraping-ai-agents
22. Top 10 AI Data Scraping Tools for RAG Pipelines (2026) | Fast.io, accessed February 21, 2026, https://fast.io/resources/top-10-ai-data-scraping-tools/
23. Malicious Chrome extensions harvest AI conversations from 900K users - Mishcon de Reya, accessed February 21, 2026, https://www.mishcon.com/news/malicious-chrome-extensions-harvest-ai-conversations-from-900k-users
24. 900,000 Users Hit as Chrome Extensions Steal AI Chat Data | eSecurity Planet, accessed February 21, 2026, https://www.esecurityplanet.com/threats/900000-users-hit-as-chrome-extensions-steal-ai-chat-data/
25. Two Chrome Extensions Caught Stealing ChatGPT and DeepSeek Chats from 900,000 Users - The Hacker News, accessed February 21, 2026, https://thehackernews.com/2026/01/two-chrome-extensions-caught-stealing.html
26. 900K Users Compromised: Chrome Extensions Steal ChatGPT and DeepSeek Conversations - OX Security, accessed February 21, 2026, https://www.ox.security/blog/malicious-chrome-extensions-steal-chatgpt-deepseek-conversations/
27. Chrome Extensions Stole 900K AI Conversations, Is Your SaaS Environment Next?, accessed February 21, 2026, https://www.reco.ai/blog/chrome-extensions-stole-900k-ai-conversations-is-your-saas-environment-next
28. Best AI Context Engines for Personalized Outreach in 2026 - Octave HQ, accessed February 21, 2026, https://www.octavehq.com/post/best-ai-context-engines-for-personalized-outreach-in-2026
29. A Comprehensive Survey on Context-Aware Multi-Agent Systems: Techniques, Applications, Challenges and Future Directions - arXiv.org, accessed February 21,

2026, https://arxiv.org/html/2402.01968v2
30. [2505.07842] RAN Cortex: Memory-Augmented Intelligence for Context-Aware Decision-Making in AI-Native Networks - arXiv, accessed February 21, 2026, https://arxiv.org/abs/2505.07842
31. Atlassian research: AI adoption is rising, but friction persists, accessed February 21, 2026, https://www.atlassian.com/blog/developer/developer-experience-report-2025
32. Context Switching Cost for Developers: Research & Data - Super Productivity, accessed February 21, 2026, https://super-productivity.com/blog/context-switching-costs-for-developers/
33. The Cost of Context Switching (and How To Avoid It) - Work Life by Atlassian, accessed February 21, 2026, https://www.atlassian.com/blog/loom/cost-of-context-switching
34. The AI Productivity Paradox Research Report - Faros AI, accessed February 21, 2026, https://www.faros.ai/blog/ai-software-engineering
35. The Hidden Cost of Context Switching - BasicOps, accessed February 21, 2026, https://www.basicops.com/cb-articles/the-hidden-cost-of-context-switching-cc4za
36. 2026 data predictions: Scaling AI agents via contextual intelligence - SiliconANGLE, accessed February 21, 2026, https://siliconangle.com/2026/01/18/2026-data-predictions-scaling-ai-agents-via-contextual-intelligence/
37. Artificial Intelligence (AI) Software Market Report 2026 - The Business Research Company, accessed February 21, 2026, https://www.thebusinessresearchcompany.com/report/artificial-intelligence-ai-software-global-market-report
38. AI Agents Market Size And Share | Industry Report, 2033 - Grand View Research, accessed February 21, 2026, https://www.grandviewresearch.com/industry-analysis/ai-agents-market-report
39. What to Expect From AI in 2026: Personal Agents, Mega Alliances, and the Gigawatt Ceiling, accessed February 21, 2026, https://www.goldmansachs.com/insights/articles/what-to-expect-from-ai-in-2026-personal-agents-mega-alliances
40. Peak XV closes $1.3B fund, bets big on India's AI boom, accessed February 21, 2026, https://www.techbuzz.ai/articles/peak-xv-closes-1-3b-fund-bets-big-on-india-s-ai-boom
41. Peak XV's Bold $1.3B Gamble: Doubling Down on AI as Global VC Titans Clash in India's Booming Tech Arena, accessed February 21, 2026, https://cryptorank.io/news/feed/75c87-peak-xv-venture-capital-india-ai
42. AI Data Centers Face Critical Power Crisis: How Peak XV's $15M Bet on C2i Could Solve the Bottleneck, accessed February 21, 2026, https://www.mexc.com/en-NG/news/722647
43. AI Data Centers Face Critical Power Crisis: How Peak XV's $15M Bet on C2i Could Solve the Bottleneck - CryptoRank, accessed February 21, 2026,

https://cryptorank.io/news/feed/22882-ai-data-centers-power-c2i-peak-xv
44. Infrastructure Startups funded by Y Combinator (YC) 2026, accessed February 21, 2026, https://www.ycombinator.com/companies/industry/infrastructure
45. Latest AI Startup Funding News and VC Investment Deals - 2025 - Crescendo.ai, accessed February 21, 2026, https://www.crescendo.ai/news/latest-vc-investment-deals-in-ai-startups